Báo cáo kết quả tuần 18–24/10

Đề tài: Phát hiện trùng lặp mã nguồn C/C++ và Python

Ngày 25 tháng 10 năm 2026

1 Kết quả tuần này

- Mở rộng bộ kiểm thử từ **41** lên **89 cặp**. Giữ nguyên 25 cặp trùng lặp và bổ sung **48 cặp không trùng lặp**, đưa nhóm này từ 16 lên **64 cặp** (đúng 4 lần bản cũ) cho sát tỉ lệ thực tế. Các cặp mới gồm 24 cặp "cùng bài, khác lõi", 6 cặp base–base khác bài còn thiếu và 18 cặp khác bài nhưng cùng kiểu biến đổi (`while`/`dowhile`/`scanf`/`combo`). Toàn bộ sinh tất định, có kiểm tra trùng cặp.
- Chấm **89 cặp** bằng 5 phương pháp: CodeBERT (M1), GraphCodeBERT (M2), ASTNN (M3), CPG-GNN (M4), mô hình kết hợp (M5). Ba phương pháp đọc văn bản (M1, M2, M5) nhận tối đa 512 đơn vị từ (token). MOSS (B1) chỉ có điểm trên **41 cặp gốc** (thiếu `MOSS_USER_ID`), nên mọi so sánh với B1 được thực hiện trên **41 cặp chung**.
- **Kết quả chính:** cả 5 mô hình bắt đúng **25/25** cặp trùng lặp (recall 1,000), nhưng vẫn báo "trùng lặp" cho rất nhiều cặp không trùng: M1 23/64, M2 22/64, M5 26/64, M3 50/64, M4 62/64. F1 lần lượt: M2 **0,694**, M1 0,685, M5 0,658, M3 0,500, M4 0,446. Khoảng cách điểm clone/non-clone của M3 (0,197) và M4 (0,093) quá nhỏ, nên hai mô hình này gần như không phân biệt được hai lớp.
- Trên **41 cặp chung** (đủ 6 phương pháp): F1 của M2 0,909, M1 0,893, M5 0,877, M3 0,794, M4 0,758, B1 0,400. Như vậy thứ hạng không đổi, nhưng điểm tuyệt đối giảm rõ khi bộ test có nhiều cặp không trùng lặp hơn: các mô hình đang bám vào khung chung của hai chương trình thay vì so sánh lõi.

2 Bộ test

Nhóm	Nội dung	Tệp	Dòng	Cặp
P1–P4	4 bài cơ bản: tổng $1..n$, ước chung lớn nhất, max mảng, số nguyên tố	24	10–18	36
P5	Danh sách liên kết + hướng đối tượng: `struct` + hàm ↔ `class`	6	59–70	10
P6	Danh sách liên kết có thứ tự, 5 hàm, có bản `class`	6	83–93	9
Khác bài	hai bài khác đề (kiểm tra âm tính), gồm 6 cặp base–base và 18 cặp cùng biến đổi	–	–	34
Tổng	25 cặp trùng lặp + 64 cặp không trùng lặp	36	–	**89**

3 Kết quả test và đánh giá

3.1 Chi tiết 41 cặp gốc (đủ 6 phương pháp)

Bảng dưới giữ nguyên 41 cặp của bộ cũ để so sánh trực tiếp với báo cáo trước. Các ô đỏ là kết luận sai tại ngưỡng đóng băng của từng phương pháp.

Cặp	Nhãn	B1	M1	M2	M3	M4	M5
Nhóm P1: Tính tổng $1..n$ (5 cặp)							
P1_while	trùng lặp	0,000	1,000	1,000	0,945	0,973	1,000
P1_dowhile	trùng lặp	0,000	1,000	1,000	0,760	0,973	1,000
P1_scanf	trùng lặp	0,000	1,000	1,000	0,931	0,973	1,000
P1_combo	trùng lặp	0,000	1,000	1,000	0,812	0,915	1,000
P1_nonclone_fact	khác	0,000	0,999	0,991	0,960	0,971	0,723
Nhóm P2: Ước chung lớn nhất (5 cặp)							

Cặp	Nhãn	B1	M1	M2	M3	M4	M5
P2_for	trùng lặp	0,000	1,000	1,000	0,955	0,972	1,000
P2_dowhile	trùng lặp	0,000	1,000	1,000	0,909	0,973	1,000
P2_scanf	trùng lặp	0,000	1,000	1,000	0,834	0,968	1,000
P2_combo	trùng lặp	0,000	1,000	1,000	0,796	0,973	1,000
P2_nonclone_digitsum	khác	0,000	0,000	0,001	0,799	0,973	0,000
Nhóm P3: Tìm giá trị lớn nhất của mảng (5 cặp)							
P3_while	trùng lặp	0,000	1,000	1,000	0,952	0,973	1,000
P3_dowhile	trùng lặp	0,000	1,000	1,000	0,479	0,973	1,000
P3_scanf	trùng lặp	0,000	1,000	1,000	0,920	0,967	1,000
P3_combo	trùng lặp	0,000	1,000	1,000	0,932	0,841	1,000
P3_nonclone_sum	khác	0,000	0,000	0,005	0,852	0,973	0,000
Nhóm P4: Kiểm tra số nguyên tố (5 cặp)							
P4_while	trùng lặp	0,000	1,000	1,000	0,892	0,967	1,000
P4_dowhile	trùng lặp	0,000	1,000	1,000	0,863	0,967	1,000
P4_scanf	trùng lặp	0,000	1,000	1,000	0,895	0,967	1,000
P4_combo	trùng lặp	0,000	1,000	1,000	0,931	0,967	0,994
P4_nonclone_square	khác	0,000	1,000	1,000	0,864	0,967	1,000
Khác bài P1–P4: Kiểm tra âm tính (6 cặp)							
X_P1P2	khác	0,000	0,000	0,000	0,868	0,960	0,000
X_P1P3	khác	0,000	0,000	0,174	0,931	0,973	0,000
X_P1P4	khác	0,000	0,000	0,000	0,618	0,914	0,000
X_P2P3	khác	0,000	0,000	0,000	0,913	0,873	0,000
X_P2P4	khác	0,000	0,000	0,023	0,777	0,660	0,027
X_P3P4	khác	0,000	0,000	0,000	0,784	0,660	0,000
Nhóm P5: Danh sách liên kết cơ bản + struct/class (8 cặp)							
P5_struct_class	trùng lặp	0,000	1,000	1,000	0,915	0,973	1,000
P5_while	trùng lặp	0,540	1,000	1,000	0,993	0,973	1,000
P5_scanf	trùng lặp	0,710	1,000	1,000	0,995	0,973	1,000
P5_combo	trùng lặp	0,150	1,000	1,000	0,994	0,973	1,000
P5_class_combo	trùng lặp	0,000	1,000	1,000	0,907	0,973	1,000
P5_nonclone_palindrome	khác	0,000	0,002	0,000	0,270	0,743	0,002
X_P1P5	khác	0,000	1,000	1,000	0,179	0,742	1,000
X_P2P5	khác	0,000	0,000	0,000	0,375	0,710	0,000
Nhóm P6: Danh sách liên kết có thứ tự, ≥ 80 dòng (7 cặp)							
P6_struct_class	trùng lặp	0,400	1,000	1,000	0,970	0,967	1,000
P6_while	trùng lặp	0,440	1,000	1,000	0,992	0,969	1,000
P6_scanf	trùng lặp	0,770	1,000	1,000	0,996	0,967	1,000
P6_combo	trùng lặp	0,200	1,000	1,000	0,995	0,967	1,000
P6_nonclone_filter	khác	0,400	1,000	0,998	0,994	0,973	0,866
X_P5P6	khác	0,510	1,000	1,000	0,992	0,973	0,997
X_P5P6_class	khác	0,360	0,997	0,554	0,968	0,973	0,020
Ngưỡng		0,010	0,210	0,660	0,420	0,470	0,010
Số lỗi P1	5 cặp	4	1	1	1	1	1
Số lỗi P2	5 cặp	4	0	0	1	1	0
Số lỗi P3	5 cặp	4	0	0	1	1	0
Số lỗi P4	5 cặp	4	1	1	1	1	1
Số lỗi khác bài	6 cặp	0	0	0	6	6	1
Số lỗi P5	8 cặp	2	1	1	0	3	1
Số lỗi P6	7 cặp	3	3	2	3	3	3
Tổng số lỗi	**41 cặp**	**21**	**6**	**5**	**13**	**16**	**7**

3.2 Tổng hợp mean ± SD trên toàn bộ 89 cặp

Điểm trung bình ± SD của điểm thô đo độ phân tán giữa các cặp; F1 ± SD là độ bất định ước lượng bằng bootstrap 2.000 lần trên các cặp (ngưỡng đóng băng: B1 0,010; M1 0,210; M2 0,660; M3 0,420; M4 0,470; M5 0,010).

Bảng 2: Toàn bộ 89 cặp có điểm (M1–M5); $\bar{s}_{cl}/\bar{s}_{nc}$ là điểm trung bình clone/non-clone ± SD

Mô hình	n	Acc	P	R	F1	F1 ± SD	$\bar{s}_{cl}$	$\bar{s}_{nc}$	Khoảng cách
M1	89	0,742	0,521	1,000	0,685	0,680 ± 0,064	1,000 ± 0,000	0,350 ± 0,472	0,650
M2	89	0,753	0,532	1,000	0,694	0,689 ± 0,064	1,000 ± 0,000	0,362 ± 0,457	0,638
M3	89	0,438	0,333	1,000	0,500	0,496 ± 0,061	0,903 ± 0,110	0,706 ± 0,263	0,197
M4	89	0,303	0,287	1,000	0,446	0,442 ± 0,059	0,963 ± 0,028	0,870 ± 0,149	0,093
M5	89	0,708	0,490	1,000	0,658	0,653 ± 0,064	1,000 ± 0,001	0,246 ± 0,406	0,753

Bảng 3: Điểm trung bình ± SD theo nhóm cặp (nhóm clone dùng điểm clone, nhóm non-clone dùng điểm non-clone)

Mô hình	Clone dễ	Clone khó	Không clone khó	Không clone dễ
Số cặp	16	9	30	34
M1	1,000 ± 0,000	1,000 ± 0,000	0,477 ± 0,499	0,238 ± 0,423
M2	1,000 ± 0,000	1,000 ± 0,000	0,488 ± 0,491	0,251 ± 0,400
M3	0,895 ± 0,128	0,917 ± 0,072	0,728 ± 0,231	0,686 ± 0,290
M4	0,971 ± 0,003	0,950 ± 0,045	0,927 ± 0,094	0,820 ± 0,170
M5	1,000 ± 0,000	0,999 ± 0,002	0,381 ± 0,452	0,127 ± 0,323

Bảng 4: Số lỗi (FP + FN) theo nhóm

Mô hình	Clone dễ	Clone khó	Không clone khó	Không clone dễ
Số cặp	16	9	30	34
M1	0	0	15	8
M2	0	0	14	8
M3	0	0	26	24
M4	0	0	30	32
M5	0	0	16	10

Hai bảng theo nhóm cho thấy lỗi hoàn toàn nằm ở phía non-clone: mọi mô hình đều đúng 25/25 cặp clone, kể cả 9 cặp "clone khó". Nhóm "không clone khó" (cùng bài, khác lõi) khó hơn nhóm "không clone dễ" (khác bài) với M1, M2, M5, đúng như kỳ vọng; riêng M3 và M4 lại cho điểm non-clone rất cao (0,706 và 0,870) nên gần như mất khả năng phân biệt.

3.3 So sánh công bằng trên 41 cặp chung

B1 thấp hơn hẳn vì MOSS chỉ nhận ra 7/25 cặp clone (recall 0,280) trên 41 cặp gốc; điểm tuyệt đối của B1 không so sánh được với 89 cặp. Trên cùng 41 cặp, M2 vẫn dẫn đầu (F1 0,909), tiếp theo là M1 (0,893) và M5 (0,877); khoảng cách giữa ba mô hình này chỉ 0,032, trong khi SD bootstrap khoảng 0,04–0,05, nên chưa thể kết luận chúng khác nhau có ý nghĩa thống kê.

Bảng 5: 41 cặp có điểm của cả 6 phương pháp; $\bar{s}_{cl}/\bar{s}_{nc}$ là điểm trung bình clone/non-clone ± SD

Mô hình	n	Acc	P	R	F1	F1 ± SD	$\bar{s}_{cl}$	$\bar{s}_{nc}$	Khoảng cách
B1	41	0,488	0,700	0,280	0,400	0,393 ± 0,106	0,128 ± 0,241	0,079 ± 0,173	0,049
M1	41	0,854	0,806	1,000	0,893	0,890 ± 0,044	1,000 ± 0,000	0,375 ± 0,500	0,625
M2	41	0,878	0,833	1,000	0,909	0,906 ± 0,042	1,000 ± 0,000	0,359 ± 0,466	0,641
M3	41	0,683	0,658	1,000	0,794	0,790 ± 0,055	0,903 ± 0,110	0,759 ± 0,261	0,144
M4	41	0,610	0,610	1,000	0,758	0,754 ± 0,058	0,963 ± 0,028	0,878 ± 0,126	0,086
M5	41	0,829	0,781	1,000	0,877	0,874 ± 0,047	1,000 ± 0,001	0,290 ± 0,442	0,710

4 Kế hoạch tuần tới

5 Phụ lục: mã nguồn minh họa (P6)

`p6_base_list.cpp` 84 dòng — bản gốc: `struct Node` + 5 hàm tự do

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node *next;
};

Node* insertSorted(Node *head, int v) {
    Node *node = new Node;
    node->val = v;
    node->next = NULL;
    if (head == NULL || v <= head->val) {
        node->next = head;
        return node;
    }
    Node *p = head;
    while (p->next != NULL && p->next->val <= v) {
        p = p->next;
    }
    node->next = p->next;
    p->next = node;
    return head;
}

int removeDuplicates(Node *head) {
    int removed = 0;
    Node *p = head;
    while (p != NULL && p->next != NULL) {
        if (p->val == p->next->val) {
            Node *dup = p->next;
            p->next = dup->next;
            delete dup;
            removed = removed + 1;
        } else {
            p = p->next;
        }
    }
    return removed;
}

long long sumList(Node *head) {
    long long total = 0;
    Node *p = head;
    while (p != NULL) {
        total = total + p->val;
        p = p->next;
    }
    return total;
}

void printList(Node *head) {
    Node *p = head;
    while (p != NULL) {
        cout << p->val << " ";
        p = p->next;
    }
    cout << endl;
}

void freeList(Node *head) {
    while (head != NULL) {
        Node *nxt = head->next;
        delete head;
        head = nxt;
    }
}

int main() {
    int n;
    cin >> n;
    Node *head = NULL;
    for (int i = 0; i < n; i++) {
```

```
        int x;
        cin >> x;
        head = insertSorted(head, x);
    }
    printList(head);
    cout << "removed " << removeDuplicates(head) << endl;
    printList(head);
    cout << "sum " << sumList(head) << endl;
    freeList(head);
    return 0;
}
```

p6_v_while.cpp

93 dòng — trùng lặp: `while` → `for`/`do-while`, đổi tên biến

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node *next;
};

Node* insertSorted(Node *head, int v) {
    Node *fresh = new Node;
    fresh->val = v;
    fresh->next = NULL;
    if (head == NULL || v <= head->val) {
        fresh->next = head;
        return fresh;
    }
    Node *cur = head;
    for (; cur->next != NULL; cur = cur->next) {
        if (cur->next->val > v) {
            break;
        }
    }
    fresh->next = cur->next;
    cur->next = fresh;
    return head;
}

int removeDuplicates(Node *head) {
    int cnt = 0;
    Node *cur = head;
    while (cur != NULL) {
        if (cur->next == NULL) {
            break;
        }
        if (cur->val != cur->next->val) {
            cur = cur->next;
            continue;
        }
        Node *dup = cur->next;
        cur->next = dup->next;
        delete dup;
        cnt = cnt + 1;
    }
    return cnt;
}

long long sumList(Node *head) {
    long long acc = 0;
    Node *cur = head;
    do {
        if (cur == NULL) {
            break;
        }
        acc = acc + cur->val;
        cur = cur->next;
    } while (cur != NULL);
    return acc;
}

void printList(Node *head) {
    for (Node *cur = head; cur != NULL; cur = cur->next) {
        cout << cur->val << " ";
    }
    cout << endl;
}

void freeList(Node *head) {
    Node *cur = head;
    while (cur != NULL) {
        Node *nxt = cur->next;
        delete cur;
        cur = nxt;
    }
}

int main() {
    int n;
    cin >> n;
    Node *head = NULL;
    int i = 0;
    while (i < n) {
        int x;
        cin >> x;
        head = insertSorted(head, x);
        i = i + 1;
    }
```

```
    printList(head);
    cout << "removed " << removeDuplicates(head) << endl;
    printList(head);
    cout << "sum " << sumList(head) << endl;
    freeList(head);
    return 0;
}
```

p6_v_scanf.cpp

83 dòng — trùng lặp: `cin/cout` → `scanf/printf`

```
#include <cstdio>

struct Node {
    int val;
    Node *next;
};

Node* insertSorted(Node *head, int v) {
    Node *node = new Node;
    node->val = v;
    node->next = NULL;
    if (head == NULL || v <= head->val) {
        node->next = head;
        return node;
    }
    Node *p = head;
    while (p->next != NULL && p->next->val <= v) {
        p = p->next;
    }
    node->next = p->next;
    p->next = node;
    return head;
}

int removeDuplicates(Node *head) {
    int removed = 0;
    Node *p = head;
    while (p != NULL && p->next != NULL) {
        if (p->val == p->next->val) {
            Node *dup = p->next;
            p->next = dup->next;
            delete dup;
            removed = removed + 1;
        } else {
            p = p->next;
        }
    }
    return removed;
}

long long sumList(Node *head) {
    long long total = 0;
    Node *p = head;
    while (p != NULL) {
        total = total + p->val;
        p = p->next;
    }
    return total;
}

void printList(Node *head) {
    Node *p = head;
    while (p != NULL) {
        printf("%d ", p->val);
        p = p->next;
    }
    printf("\n");
}

void freeList(Node *head) {
    while (head != NULL) {
        Node *nxt = head->next;
        delete head;
        head = nxt;
    }
}

int main() {
    int n;
    scanf("%d", &n);
    Node *head = NULL;
    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);
        head = insertSorted(head, x);
    }
    printList(head);
    printf("removed %d\n", removeDuplicates(head));
    printList(head);
    printf("sum %lld\n", sumList(head));
    freeList(head);
    return 0;
}
```

p6_v_combo.cpp 87 dòng — trùng lặp khó: đảo điều kiện + biến thừa

```cpp
#include <cstdio>

struct Node {
    int val;
    Node *next;
};

Node* addNode(Node *head, int v) {
    Node *fresh = new Node;
    fresh->val = v;
    fresh->next = NULL;
    if (!(head != NULL && v > head->val)) {
        fresh->next = head;
        return fresh;
    }
    Node *cur = head;
    while (!(cur->next == NULL || cur->next->val > v)) {
        cur = cur->next;
    }
    fresh->next = cur->next;
    cur->next = fresh;
    return head;
}

int dropRepeat(Node *head) {
    int cnt = 0;
    int unused = 7;
    Node *cur = head;
    do {
        if (cur == NULL || cur->next == NULL) {
            break;
        }
        if (cur->val != cur->next->val) {
            cur = cur->next;
            continue;
        }
        Node *dup = cur->next;
        cur->next = dup->next;
        delete dup;
        cnt = cnt + 1;
    } while (cur != NULL);
    return cnt + unused * 0;
}

long long accSum(Node *head) {
    long long acc = 0;
    Node *cur = head;
    while (!(cur == NULL)) {
        acc = acc + cur->val;
        cur = cur->next;
    }
    return acc;
}

void show(Node *head) {
    for (Node *cur = head; cur != NULL; cur = cur->next) {
        printf("%d ", cur->val);
    }
    printf("\n");
}

void release(Node *head) {
    while (!(head == NULL)) {
        Node *nxt = head->next;
        delete head;
        head = nxt;
    }
}

int main() {
    int n;
    scanf("%d", &n);
    Node *head = NULL;
    int i = 0;
    while (i < n) {
        int x;
        scanf("%d", &x);
        head = addNode(head, x);
        i = i + 1;
    }
    show(head);
    printf("removed %d\n", dropRepeat(head));
    show(head);
    printf("sum %lld\n", accSum(head));
    release(head);
    return 0;
}
```

p6_v_class.cpp 86 dòng — trùng lặp: `class IntList` + hàm hủy, cùng hành vi

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node *next;
};

class IntList {
private:
    Node *head;
public:
    IntList() {
        head = NULL;
    }
    ~IntList() {
        clear();
    }
    void insert(int v) {
        Node *node = new Node;
        node->val = v;
        node->next = NULL;
        if (head == NULL || v <= head->val) {
            node->next = head;
            head = node;
            return;
        }
        Node *p = head;
        while (p->next != NULL && p->next->val <= v) {
            p = p->next;
        }
        node->next = p->next;
        p->next = node;
    }
    int dedup() {
        int removed = 0;
        Node *p = head;
        while (p != NULL && p->next != NULL) {
            if (p->val == p->next->val) {
                Node *dup = p->next;
                p->next = dup->next;
                delete dup;
                removed = removed + 1;
            } else {
                p = p->next;
            }
        }
        return removed;
    }
    long long total() const {
        long long sum = 0;
        for (Node *p = head; p != NULL; p = p->next) {
            sum = sum + p->val;
        }
        return sum;
    }
    void show() const {
        for (Node *p = head; p != NULL; p = p->next) {
            cout << p->val << " ";
        }
        cout << endl;
    }
    void clear() {
        while (head != NULL) {
            Node *nxt = head->next;
            delete head;
            head = nxt;
        }
    }
};

int main() {
    int n;
    cin >> n;
    IntList list;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        list.insert(x);
    }
    list.show();
    cout << "removed " << list.dedup() << endl;
    list.show();
    cout << "sum " << list.total() << endl;
    return 0;
}
```

p6_n_filter.cpp 88 dòng — không trùng lặp: cùng khung, khác lõi ở giữa tệp

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int val;
    Node *next;
};

Node* pushBack(Node *head, int v) {
    Node *node = new Node;
    node->val = v;
    node->next = NULL;
    if (head == NULL) {
        return node;
    }
    Node *p = head;
    while (p->next != NULL) {
        p = p->next;
    }
    p->next = node;
    return head;
}

int removeGreater(Node *&head, int limit) {
    int removed = 0;
    while (head != NULL && head->val > limit) {
        Node *old = head;
        head = head->next;
        delete old;
        removed = removed + 1;
    }
    Node *p = head;
    while (p != NULL && p->next != NULL) {
        if (p->next->val > limit) {
            Node *old = p->next;
            p->next = old->next;
            delete old;
            removed = removed + 1;
        } else {
            p = p->next;
        }
    }
    return removed;
}

int countOdd(Node *head) {
    int odd = 0;
    for (Node *p = head; p != NULL; p = p->next) {
        if (p->val % 2 != 0) {
            odd = odd + 1;
        }
    }
    return odd;
}

void printList(Node *head) {
    for (Node *p = head; p != NULL; p = p->next) {
        cout << p->val << " ";
    }
    cout << endl;
}

void freeList(Node *head) {
    while (head != NULL) {
        Node *nxt = head->next;
        delete head;
        head = nxt;
    }
}

int main() {
    int n;
    cin >> n;
    Node *head = NULL;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        head = pushBack(head, x);
    }
    printList(head);
    int limit;
    cin >> limit;
    cout << "removed " << removeGreater(head, limit) << endl;
    printList(head);
    cout << "odd " << countOdd(head) << endl;
    freeList(head);
    return 0;
}
```